

대용량 HTTP 요청 처리

메모리 효율적인 Async Streaming 패턴

목표: $O(N) \rightarrow O(\text{concurrency})$ 메모리 최적화

Pain Point: 현실의 문제

10만 건 API 호출을 해야 한다면?

```
# 가장 단순한 방법
urls = [f"https://api.example.com/item/{i}" for i in range(100_000)]
results = await asyncio.gather(*[fetch(url) for url in urls])
```

결과

지표	값
Task 객체	100,000개 동시 생성
메모리	수 GB
서버	429 Too Many Requests

오늘 해결할 것

Before

- 메모리: $O(N)$
- 모든 결과 대기 후 처리
- 파이프라인 연결 불가

After

- 메모리: $O(\text{concurrency})$
- 결과 즉시 스트리밍
- Stage 체이닝 가능

[100,000 items] → [10 workers] → [결과 하나씩 yield]
 ↑
 동시에 10개만 처리

Part 1: V1 → V2 빠른 리뷰

V1: Semaphore 방식

```
async def fetch_all(urls, concurrency=10):  
    sem = asyncio.Semaphore(concurrency)  
  
    async def fetch_with_sem(url):  
        async with sem:  
            async with aiohttp.ClientSession() as session:  
                async with session.get(url) as resp:  
                    return await resp.json()  
  
    tasks = [fetch_with_sem(url) for url in urls] # ← 여기가 문제!  
    return await asyncio.gather(*tasks)
```

V1의 문제

```
urls = [url1, url2, ..., url100000]
      ↓
tasks = [Task1, Task2, ..., Task100000] ← 10만개 동시 생성!
      ↓
await gather(*tasks) ← 전부 완료될 때까지 대기
```

문제점

- 메모리: Task 객체 N개 = $O(N)$
- 백프레셔 없음: 생산 속도 제어 불가
- 백프레셔 정의: 소비자가 처리할 수 있는 속도보다 생산자가 빠를 때, 생산자를 늦추는 메커니즘
- 스트리밍 불가: 전체 완료 후에야 결과 접근

V2: Queue + Worker 방식

핵심 구조

```

graph LR
    P[Producer] -- put() --> Q["[Queue maxsize=10]"]
    Q -- get() --> W[Workers (10개)]
    W -- 백프레시 동작 --> Q

```

개선점

- 메모리: $O(\text{concurrency})$
- 백프레시: Queue가 가득 차면 Producer 대기
- 스트리밍 불가: `queue.join()` 후에야 결과 반환

V2의 한계

```
async def run(self, fn, items) -> list[Result]:  
    # ... worker 생성 ...  
    await self.queue.join() # ← 전체 완료 대기  
    return self.results     # ← 한꺼번에 반환
```

우리가 원하는 것

```
async for result in executor.run_stream(fn, items):  
    process(result) # ← 하나씩 즉시 처리!
```

Part 2: V3 Streaming 구현

V3 요구사항

1. AsyncGenerator로 결과 yield

- `async for result in run_stream(...):`

2. 메모리 효율 유지

- 동시 처리 수만큼만 메모리 사용

3. 파이프라인 연결 가능

- Stage1 출력 → Stage2 입력

4. 안전한 종료

- 모든 worker 정상 종료 보장

[라이브 코딩] 기본 구조

```
async def run_stream(self, fn, items):
    queue = asyncio.Queue(maxsize=self.concurrency * 2)
    result_queue = asyncio.Queue() # unbounded

    async def producer():
        # items를 queue에 넣기
        pass

    async def worker():
        # queue에서 꺼내서 처리, result_queue에 넣기
        pass

    # 결과 수집 및 yield
    pass
```

[라이브 코딩] Producer & Worker

```
async def producer():
    try:
        for item in items:
            await queue.put(item)
    finally:
        for _ in range(self.concurrency):
            await queue.put(_DONE) # Sentinel

async def worker():
    while True:
        item = await queue.get()
        if isinstance(item, _Sentinel):
            await result_queue.put(_DONE)
            return
        try:
            value = await fn(item)
            await result_queue.put(Result(item, value, None))
        except Exception as e:
            await result_queue.put(Result(item, None, e))
```

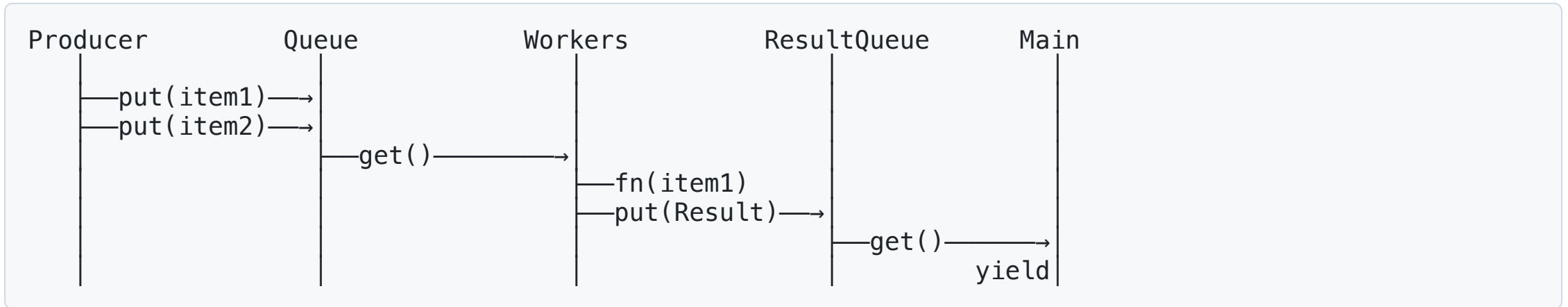
[라이브 코딩] 결과 수집

```
worker_tasks = [asyncio.create_task(worker())
                 for _ in range(self.concurrency)]
producer_task = asyncio.create_task(producer())

# N개의 종료 신호를 받을 때까지
done_count = 0
while done_count < self.concurrency:
    result = await result_queue.get()
    if isinstance(result, _Sentinel):
        done_count += 1
    else:
        yield result # ← 즉시 yield!

await producer_task
await asyncio.gather(*worker_tasks)
```

동작 원리: 시퀀스 다이어그램



Sentinel 패턴: 안전한 종료

1. Producer: items 소진
↳ put(_DONE) × N개
2. Workers: _DONE 수신
↳ put(_DONE) to result_queue
↳ return
3. Main: _DONE 카운트
↳ done_count == N → 탈출

왜 이렇게?

- 어떤 worker가 죽어도 나머지가 신호 보냄
- 데드락 방지

[라이브 데모] 메모리 측정

```
import tracemalloc

tracemalloc.start()
# QueuedExecutor 실행
peak_queued = tracemalloc.get_traced_memory()[1]
tracemalloc.stop()

tracemalloc.start()
# Semaphore 방식 실행
peak_sem = tracemalloc.get_traced_memory()[1]
tracemalloc.stop()

print(f"QueuedExecutor: {peak_queued / 1024 / 1024:.2f} MB")
print(f"Semaphore: {peak_sem / 1024 / 1024:.2f} MB")
```

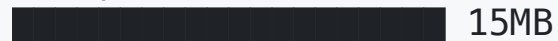
메모리 비교 결과

10,000 items 처리

방식	Peak 메모리
Semaphore	15.18 MB
QueuedExecutor	1.85 MB

메모리 사용 패턴

Semaphore:



↑ N개 Task

QueuedExecutor:



↑ 10개 Worker만

8배 메모리 절감

Part 3: 파이프라인 체이닝

배치 vs 스트리밍 파이프라인

배치 (직렬)

[Stage1 전체 완료]

↓

[Stage2 시작]

총 시간 = $S1 + S2$

스트리밍 (병렬)

[Stage1 ↓↓↓]

[Stage2 ↑↑↑]

동시 진행

총 시간 $\approx \max(S1, S2)$

체이닝 코드 패턴

```
stage1 = QueuedExecutor(concurrency=10)
stage2 = QueuedExecutor(concurrency=10)

# Stage1의 스트림을 Stage2의 입력으로 연결
async def stage1_results():
    async for result in stage1.run_stream(fetch_raw, ids):
        if result.ok:
            yield result.value # AsyncGenerator

# Stage2가 Stage1을 소비
async for result in stage2.run_stream(enrich, stage1_results()):
    process(result)
```

벤치마크 결과 (1000 items)

지표	스트리밍	배치	차이
첫 결과까지	0.017s	1.105s	65배 빠름
전체 시간	1.06s	1.67s	36% 단축
메모리	0.06 MB	1.25 MB	95% 절감

왜?

- Stage1이 결과 내는 즉시 Stage2가 소비
- 중간 결과 전체를 메모리에 안 쌓음

ETL 실전 패턴

```
async def etl():
    fetcher = QueuedExecutor(concurrency=20)    # I/O bound
    transformer = QueuedExecutor(concurrency=5) # rate limit

    # 스트리밍 파이프라인
    async def fetch_stream():
        async for r in fetcher.run_stream(fetch, urls):
            if r.ok: yield r.value

    async def transform_stream():
        async for r in transformer.run_stream(parse, fetch_stream()):
            if r.ok: yield r.value

    # 배치로 저장
    async for batch in batch_iter(transform_stream(), size=1000):
        await db.bulk_insert(batch)
```

Part 4: 정리

언제 무엇을 쓸까?

```
메모리 제한 심함?  
└ Yes → 스트리밍  
└ No  
    └ 외부 시스템이 벌크 API?  
        └ Yes → 배치  
        └ No  
            └ 실시간성 필요?  
                └ Yes → 스트리밍  
                └ No → 배치
```

핵심 포인트 3가지

1. 메모리는 O(concurrency)

Queue + Worker 패턴으로 동시 처리 수만큼만 사용

2. Sentinel로 안전한 종료

모든 worker가 종료 신호 → 데드락 방지

3. AsyncGenerator로 파이프라인

Stage 체이닝 → 시간 36% 단축, 메모리 95% 절감

Q&A

코드: `learning_pub_sub.py`

문서: `pything_7_playground.md`